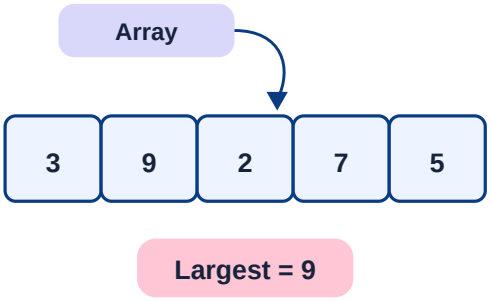
Program
01

Find the Largest Element in an Array

Topic
Arrays

Problem Statement

Given an array of integers, find and return the largest element present in the array.



Approach

1. Assume the first element is the largest.
2. Traverse the array from left to right.
3. If an element is greater than largest, update largest.
4. After the loop, largest holds the maximum value.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

largest = arr[0]
for x in arr:
    if x > largest:
        largest = x

print("Largest element is:", largest)
```

Input

```
5
3 9 2 7 5
```

Line 1: n | Line 2: array elements

Output

```
Largest element is: 9
```

Dry Run / Illustration

Step	Current	Largest (Before)	Largest (After)
1	3	3	3
2	9	3	9
3	2	9	9
4	7	9	9
5	5	9	9

Key Takeaways

- A single pass is enough - no sorting needed.
- Start with arr[0], not 0, so negative arrays also work.
- Python shortcut is max(arr), but interviews expect the loop.

Time and Space Complexity

- **Time Complexity:** O(n)
  We traverse the array once.
- **Space Complexity:** O(1)
  Only one extra variable is used.

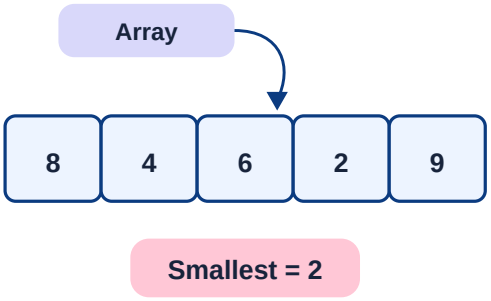
Program
02

Find the Smallest Element in an Array

Topic
Arrays

Problem Statement

Given an array of integers, find and return the smallest element present in the array.



Approach

1. Assume the first element is the smallest.
2. Traverse the array from left to right.
3. If an element is smaller than smallest, update smallest.
4. After the loop, smallest holds the minimum value.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

smallest = arr[0]
for x in arr:
    if x < smallest:
        smallest = x

print("Smallest element is:", smallest)
```

Input

```
5
8 4 6 2 9
```

Line 1: n | Line 2: array elements

Output

```
Smallest element is: 2
```

Dry Run / Illustration

Step	Current	Smallest (Before)	Smallest (After)
1	8	8	8
2	4	8	4
3	6	4	4
4	2	4	2
5	9	2	2

Key Takeaways

- Same logic as the largest element - only the comparison flips.
- Do both in one loop to find min and max together.
- Never initialise with 0 or a fixed number - use arr[0].

Time and Space Complexity

- **Time Complexity:** O(n)
  Each element is checked once.
- **Space Complexity:** O(1)
  Only one extra variable is used.

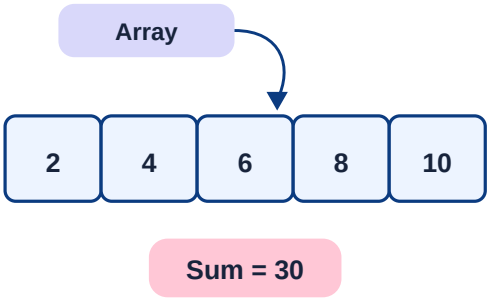
Program
03

Find the Sum of all Elements in an Array

Topic
Arrays

Problem Statement

Given an array of integers, find and return the sum of all elements present in the array.



Approach

1. Initialize a variable total = 0.
2. Traverse the array from the first element to the last.
3. Add each element to the total variable.
4. After traversing the entire array, total contains the sum of all elements.

Python Program

```python
n = int(input())
arr = list(map(int, input().split()))

total = 0
for x in arr:
    total += x

print("Sum of all elements is:", total)
```

Input

```
5
2 4 6 8 10
```

Line 1: n | Line 2: array elements

Output

```
Sum of all elements is: 30
```

Dry Run / Illustration

Step	Current Element	Sum (Before)	Sum (After)
Initial	-	0	0
1	2	0	2
2	4	2	6
3	6	6	12
4	8	12	20
5	10	20	30

Key Takeaways

- We use a simple linear traversal to add all elements.
- A basic yet important problem often asked in interviews.
- Works for both positive and negative numbers. (Python also has sum(arr).)

Time and Space Complexity

- **Time Complexity:** O(n)
  We traverse the array once.
- **Space Complexity:** O(1)
  Only one variable is used.

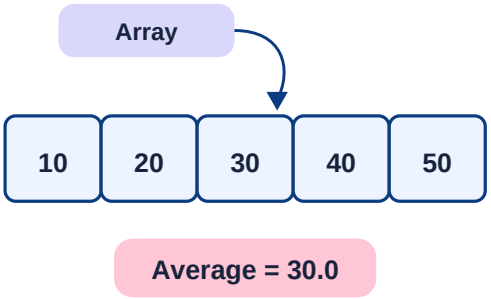
Program
04

Find the Average of Array Elements

Topic
Arrays

Problem Statement

Given an array of integers, calculate and return the average (mean) of all its elements.



Approach

1. Initialize total = 0.
2. Traverse the array and add each element to total.
3. Divide total by the number of elements n.
4. Print the result (a decimal value).

Python Program

```python
n = int(input())
arr = list(map(int, input().split()))

total = 0
for x in arr:
    total += x

average = total / n
print("Average is:", average)
```

Input

```
5
10 20 30 40 50
```

Line 1: n | Line 2: array elements

Output

```
Average is: 30.0
```

Dry Run / Illustration

Step	Current Element	Sum (Before)	Sum (After)
1	10	0	10
2	20	10	30
3	30	30	60
4	40	60	100
5	50	100	150
Avg	150 / 5	-	30.0

Key Takeaways

- Average = sum of elements / number of elements.
- In Python, "/" gives a float and "//" gives an integer division.
- Check n > 0 first to avoid ZeroDivisionError on empty input.

Time and Space Complexity

- **Time Complexity:** O(n)
  One traversal to compute the sum.
- **Space Complexity:** O(1)
  Only total and average are stored.

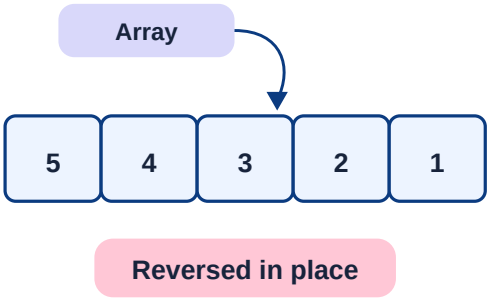
Program
05

Reverse an Array

Topic
Two Pointers

Problem Statement

Given an array of integers, reverse the order of its elements in place.



Approach

1. Place one pointer at the start (left) and one at the end (right).
2. Swap the elements at left and right.
3. Move left forward and right backward.
4. Repeat until the two pointers meet or cross.

Python Program

```python
n = int(input())
arr = list(map(int, input().split()))

left, right = 0, n - 1
while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
    right -= 1

print("Reversed array:", *arr)
```

Input

```
5
1 2 3 4 5
```

Line 1: n | Line 2: array elements

Output

```
Reversed array: 5 4 3 2 1
```

Dry Run / Illustration

Swap	left	right	Array
Start	0	4	1 2 3 4 5
1	1	3	5 2 3 4 1
2	2	2	5 4 3 2 1

Key Takeaways

- The two-pointer technique reverses with no extra array.
- Python shortcuts: arr[::-1] or arr.reverse().
- The same idea is used for strings and palindromes.

Time and Space Complexity

- **Time Complexity:** O(n)
  About n/2 swaps are performed.
- **Space Complexity:** O(1)
  Reversal happens in place.

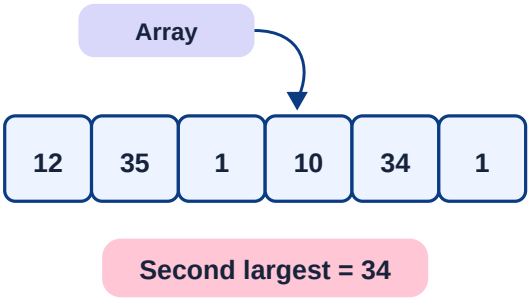
Program
06

Find the Second Largest Element

Topic
Arrays

Problem Statement

Given an array of integers, find the second largest distinct element in the array.



Approach

1. Keep two variables: first and second, both starting at minus infinity.
2. If an element is bigger than first, shift first into second and update first.
3. Else if it is between second and first, update second.
4. After one pass, second holds the answer.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

first = second = float("-inf")
for x in arr:
    if x > first:
        second = first
        first = x
    elif first > x > second:
        second = x

print("Second largest element is:", second)
```

Input

```
6
12 35 1 10 34 1
```

Line 1: n | Line 2: array elements

Output

```
Second largest element is: 34
```

Dry Run / Illustration

Element	first	second
12	12	-inf
35	35	12
1	35	12
10	35	12
34	35	34
1	35	34

Key Takeaways

- Solves it in one pass - no sorting (which would cost O(n log n)).
- The strict check first > x skips duplicates of the largest.
- If second is still -inf at the end, no second largest exists.

Time and Space Complexity

- **Time Complexity:** O(n)
  One traversal of the array.
- **Space Complexity:** O(1)
  Two variables only.

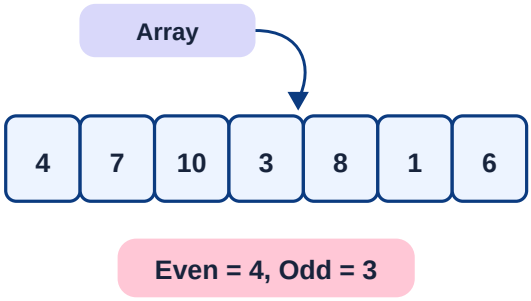
Program
07

Count Even and Odd Elements

Topic
Arrays

Problem Statement

Given an array of integers, count how many elements are even and how many are odd.



Approach

1. Initialize two counters: even = 0 and odd = 0.
2. Traverse the array element by element.
3. If x % 2 == 0, increase even; otherwise increase odd.
4. Print both counters at the end.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

even = odd = 0
for x in arr:
    if x % 2 == 0:
        even += 1
    else:
        odd += 1

print("Even count:", even)
print("Odd count:", odd)
```

Input

```
7
4 7 10 3 8 1 6
```

Line 1: n | Line 2: array elements

Output

```
Even count: 4
Odd count: 3
```

Dry Run / Illustration

Element	Type	Even	Odd
4	Even	1	0
7	Odd	1	1
10	Even	2	1
3	Odd	2	2
8	Even	3	2
1	Odd	3	3
6	Even	4	3

Key Takeaways

- The modulus operator % is the key idea here.
- Works for negative numbers too: -3 % 2 is 1 in Python.
- even + odd always equals n - a quick self-check.

Time and Space Complexity

- **Time Complexity:** O(n)
  Each element is checked once.
- **Space Complexity:** O(1)
  Two counters only.

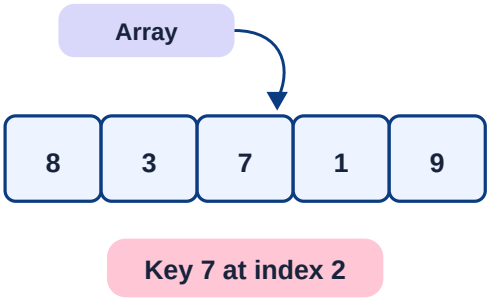
Program
08

Linear Search in an Array

Topic
Searching

Problem Statement

Given an array and a key, find the index of the key. Print -1 style message if it is absent.



Approach

1. Read the array and the key to search.
2. Traverse the array from index 0.
3. If arr[i] equals the key, store i and stop.
4. If the loop ends without a match, report not found.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))
key = int(input())

index = -1
for i in range(n):
    if arr[i] == key:
        index = i
        break

if index != -1:
    print("Element found at index:", index)
else:
    print("Element not found")
```

Input

```
5
8 3 7 1 9
7
```

Line 1: n | Line 2: elements | Line 3: key

Output

```
Element found at index: 2
```

Dry Run / Illustration

i	arr[i]	Match?
0	8	No
1	3	No
2	7	Yes - stop

Key Takeaways

- Works on any array - sorted or unsorted.
- Best case O(1) (first element), worst case O(n).
- Use binary search (Program 22) when the array is sorted.

Time and Space Complexity

- **Time Complexity:** O(n)
  Worst case checks every element.
- **Space Complexity:** O(1)
  Only an index variable is used.

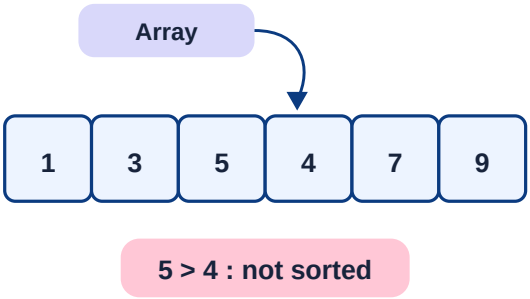
Program
09

Check if an Array is Sorted

Topic
Arrays

Problem Statement

Given an array of integers, check whether it is sorted in non-decreasing (ascending) order.



Approach

1. Assume the array is sorted.
2. Compare every element with the one before it.
3. If any element is smaller than its predecessor, it is not sorted - stop.
4. Otherwise the array is sorted.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

is_sorted = True
for i in range(1, n):
    if arr[i] < arr[i - 1]:
        is_sorted = False
        break

print("Sorted" if is_sorted else "Not sorted")
```

Input

```
6
1 3 5 4 7 9
```

Line 1: n | Line 2: array elements

Output

```
Not sorted
```

Dry Run / Illustration

i	arr[i-1]	arr[i]	Result
1	1	3	OK
2	3	5	OK
3	5	4	Not sorted

Key Takeaways

- Only adjacent pairs need to be compared.
- Stops early at the first violation.
- Arrays with 0 or 1 element are always sorted.

Time and Space Complexity

- **Time Complexity:** O(n)
  At most n - 1 comparisons.
- **Space Complexity:** O(1)
  One flag variable.

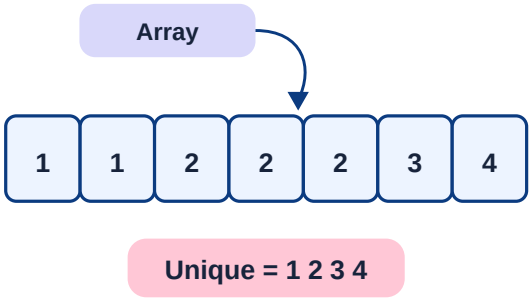
Program
10

Remove Duplicates from a Sorted Array

Topic
Two Pointers

Problem Statement

Given a sorted array, remove duplicates in place so each element appears once, and return the new length.



Approach

1. Keep a pointer j at the last unique element (start at 0).
2. Loop i from 1 to n - 1.
3. If arr[i] differs from arr[j], move j forward and copy arr[i] there.
4. The unique elements are arr[0..j]; the count is j + 1.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

j = 0
for i in range(1, n):
    if arr[i] != arr[j]:
        j += 1
        arr[j] = arr[i]

print("Unique count:", j + 1)
print("Array:", *arr[:j + 1])
```

Input

```
7
1 1 2 2 2 3 4
```

Line 1: n | Line 2: array elements

Output

```
Unique count: 4
Array: 1 2 3 4
```

Dry Run / Illustration

i	arr[i]	j	Unique part
1	1	0	1
2	2	1	1 2
3	2	1	1 2
4	2	1	1 2
5	3	2	1 2 3
6	4	3	1 2 3 4

Key Takeaways

- The array must be sorted so duplicates sit together.
- Slow pointer j builds the answer, fast pointer i scans.
- No extra array is needed - O(1) space.

Time and Space Complexity

- **Time Complexity:** O(n)
  Single pass with two pointers.
- **Space Complexity:** O(1)
  Modification is done in place.

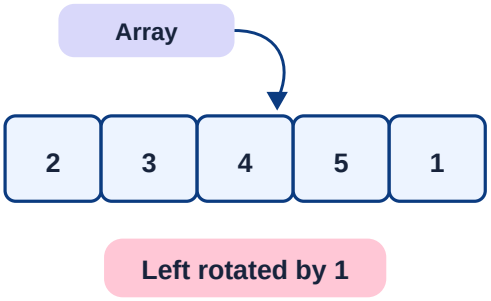
Program
11

Left Rotate an Array by One Position

Topic
Arrays

Problem Statement

Given an array, rotate it to the left by one position (the first element moves to the end).



Approach

1. Store the first element in a variable.
2. Shift every other element one place to the left.
3. Place the stored element at the last index.
4. Print the rotated array.

Python Program

```python
n = int(input())
arr = list(map(int, input().split()))

first = arr[0]
for i in range(1, n):
    arr[i - 1] = arr[i]
arr[n - 1] = first

print("After rotation:", *arr)
```

Input

```
5
1 2 3 4 5
```

Line 1: n | Line 2: array elements

Output

```
After rotation: 2 3 4 5 1
```

Dry Run / Illustration

Step	Action	Array
Save	first = 1	1 2 3 4 5
i=1	arr[0]=2	2 2 3 4 5
i=2	arr[1]=3	2 3 3 4 5
i=3	arr[2]=4	2 3 4 4 5
i=4	arr[3]=5	2 3 4 5 5
End	arr[4]=first	2 3 4 5 1

Key Takeaways

- Always save the first element before shifting.
- Rotation is a common building block for harder array problems.
- Python shortcut: arr = arr[1:] + arr[:1].

Time and Space Complexity

- **Time Complexity:** O(n)
  Every element is shifted once.
- **Space Complexity:** O(1)
  Only one temporary variable.

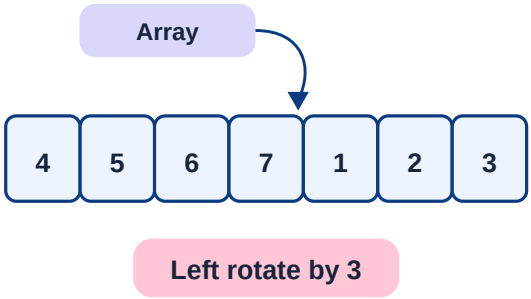
Program
12

Rotate an Array by K Positions

Topic
Arrays

Problem Statement

Given an array and a number k, rotate the array to the left by k positions.



Approach

1. Take k = k % n so the rotation count never exceeds n.
2. Reverse the first k elements.
3. Reverse the remaining n - k elements.
4. Reverse the whole array to get the final result.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))
k = int(input()) % n

def reverse(a, i, j):
    while i < j:
        a[i], a[j] = a[j], a[i]
        i += 1
        j -= 1

reverse(arr, 0, k - 1)
reverse(arr, k, n - 1)
reverse(arr, 0, n - 1)
print("After rotation:", *arr)
```

Input

```
7
1 2 3 4 5 6 7
3
```

Line 1: n | Line 2: elements | Line 3: k

Output

```
After rotation: 4 5 6 7 1 2 3
```

Dry Run / Illustration

Step	Operation	Array
Start	-	1 2 3 4 5 6 7
1	reverse(0, 2)	3 2 1 4 5 6 7
2	reverse(3, 6)	3 2 1 7 6 5 4
3	reverse(0, 6)	4 5 6 7 1 2 3

Key Takeaways

- The three-reversal trick rotates in place with O(1) extra space.
- Using k % n handles k larger than n.
- For a right rotation, reverse the whole array first.

Time and Space Complexity

- **Time Complexity:** O(n)
  Each element is moved a constant number of times.
- **Space Complexity:** O(1)
  All reversals are done in place.

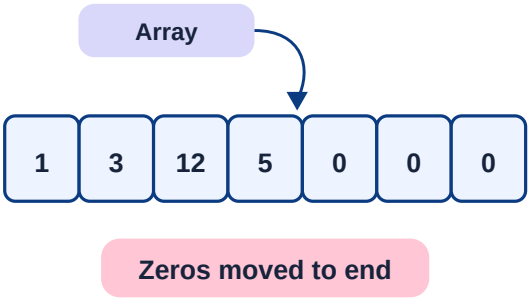
Program
13

Move All Zeros to the End

Topic
Two Pointers

Problem Statement

Given an array, move all zeros to the end while keeping the order of the non-zero elements.



Approach

1. Keep a pointer pos for the next non-zero slot (start at 0).
2. Traverse the array with index i.
3. If arr[i] is not zero, swap it with arr[pos] and move pos forward.
4. All zeros automatically end up at the back.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

pos = 0
for i in range(n):
    if arr[i] != 0:
        arr[pos], arr[i] = arr[i], arr[pos]
        pos += 1

print("Result:", *arr)
```

Input

```
7
0 1 0 3 12 0 5
```

Line 1: n | Line 2: array elements

Output

```
Result: 1 3 12 5 0 0 0
```

Dry Run / Illustration

i	arr[i]	pos	Array
0	0	0	0 1 0 3 12 0 5
1	1	1	1 0 0 3 12 0 5
2	0	1	1 0 0 3 12 0 5
3	3	2	1 3 0 0 12 0 5
4	12	3	1 3 12 0 0 0 5
5	0	3	1 3 12 0 0 0 5
6	5	4	1 3 12 5 0 0 0

Key Takeaways

- Relative order of non-zero elements is preserved.
- One pass and in place - the optimal approach.
- The same idea works for "move all X to the end".

Time and Space Complexity

- **Time Complexity:** O(n)
  Single traversal.
- **Space Complexity:** O(1)
  No extra array is used.

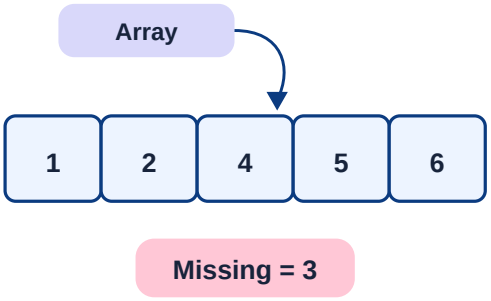
Program
14

Find the Missing Number from 1 to n

Topic
Math

Problem Statement

An array holds n - 1 distinct numbers taken from 1 to n. Find the one missing number.



Approach

1. Compute the expected sum of 1..n using n * (n + 1) / 2.
2. Compute the actual sum of the array elements.
3. The difference expected - actual is the missing number.
4. Print the missing number.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

expected = n * (n + 1) // 2
actual = sum(arr)

print("Missing number is:", expected - actual)
```

Input

```
6
1 2 4 5 6
```

Line 1: n (range 1..n) | Line 2: n - 1 numbers

Output

```
Missing number is: 3
```

Dry Run / Illustration

Step	Calculation	Result
Expected	6 x 7 / 2	21
Actual	1+2+4+5+6	18
Missing	21 - 18	3

Key Takeaways

- Uses the Gauss formula n(n+1)/2 for the sum of 1..n.
- No sorting and no extra memory required.
- XOR of all numbers also works and avoids overflow in other languages.

Time and Space Complexity

- **Time Complexity:** O(n)
  One pass to add the elements.
- **Space Complexity:** O(1)
  Only two sums are stored.

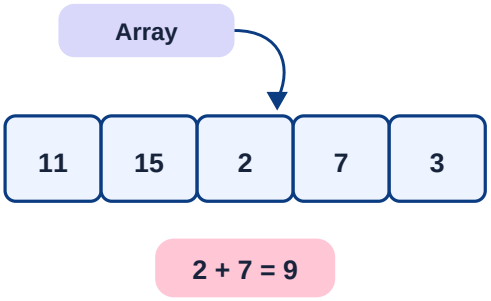
Program
15

Two Sum - Find a Pair with Given Target

Topic
Hashing

Problem Statement

Given an array and a target, find the indices of two elements whose sum equals the target.



Approach

1. Create an empty dictionary seen (value -> index).
2. For each element compute need = target - arr[i].
3. If need is already in seen, the pair is found.
4. Otherwise store arr[i] in seen and continue.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))
target = int(input())

seen = {}
for i in range(n):
    need = target - arr[i]
    if need in seen:
        print("Indices:", seen[need], i)
        break
    seen[arr[i]] = i
else:
    print("No pair found")
```

Input

```
5
11 15 2 7 3
9
```

Line 1: n | Line 2: elements | Line 3: target

Output

```
Indices: 2 3
```

Dry Run / Illustration

i	arr[i]	need	Found?
0	11	-2	No
1	15	-6	No
2	2	7	No
3	7	2	Yes (2, 3)

Key Takeaways

- A hash map turns the O(n^2) brute force into O(n).
- We look up the complement instead of scanning again.
- One of the most asked interview questions - know it well.

Time and Space Complexity

- **Time Complexity:** O(n)
  Each lookup in the dictionary is O(1).
- **Space Complexity:** O(n)
  The dictionary can hold up to n items.

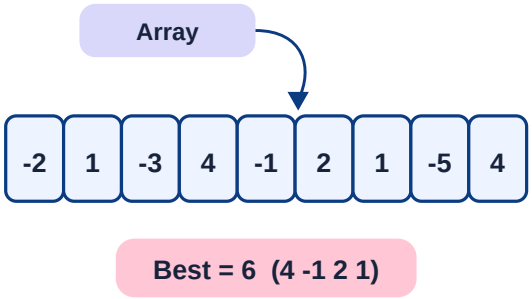
Program
16

Maximum Subarray Sum (Kadane's Algorithm)

Topic
Dynamic Prog.

Problem Statement

Given an array of integers (may contain negatives), find the largest sum of any contiguous subarray.



Approach

1. Set current and best to the first element.
2. For every next element decide: extend the subarray or start fresh.
3. current = max(arr[i], current + arr[i]).
4. best = max(best, current); print best at the end.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

current = best = arr[0]
for i in range(1, n):
    current = max(arr[i], current + arr[i])
    best = max(best, current)

print("Maximum subarray sum:", best)
```

Input

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Line 1: n | Line 2: array elements

Output

```
Maximum subarray sum: 6
```

Dry Run / Illustration

Element	current	best
-2	-2	-2
1	1	1
-3	-2	1
4	4	4
-1	3	4
2	5	5
1	6	6
-5	1	6
4	5	6

Key Takeaways

- A classic dynamic programming idea in just a few lines.
- Works even when every number is negative.
- Brute force is O(n^2); Kadane brings it down to O(n).

Time and Space Complexity

- **Time Complexity:** O(n)
  Single pass over the array.
- **Space Complexity:** O(1)
  Only current and best are stored.

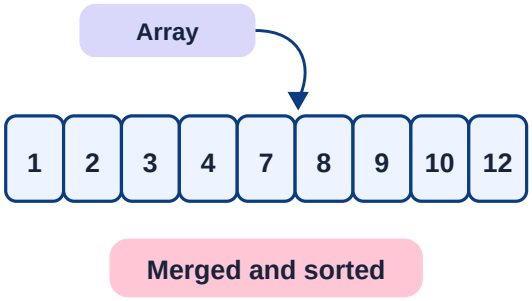
Program
17

Merge Two Sorted Arrays

Topic
Two Pointers

Problem Statement

Given two sorted arrays, merge them into a single sorted array.



Approach

1. Use pointers i and j at the start of each array.
2. Compare a[i] and b[j]; append the smaller one and advance its pointer.
3. Repeat until one array is exhausted.
4. Append the leftover elements of the other array.

Python Program

Python

```python
a = list(map(int, input().split()))
b = list(map(int, input().split()))

i = j = 0
result = []
while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        result.append(a[i])
        i += 1
    else:
        result.append(b[j])
        j += 1

result += a[i:] + b[j:]
print("Merged array:", *result)
```

Input

```
1 4 7 9
2 3 8 10 12
```

Line 1: first sorted array | Line 2: second sorted array

Output

```
Merged array: 1 2 3 4 7 8 9 10 12
```

Dry Run / Illustration

Compare	Picked	Result
1 vs 2	1	1
4 vs 2	2	1 2
4 vs 3	3	1 2 3
4 vs 8	4	1 2 3 4
7 vs 8	7	1 2 3 4 7
9 vs 8	8	1 2 3 4 7 8
9 vs 10	9	1 2 3 4 7 8 9
Rest	10 12	1 2 3 4 7 8 9 10 12

Key Takeaways

- This is the merge step of Merge Sort.
- Both pointers only move forward - no re-scanning.
- Use <= so equal elements keep their original order (stable).

Time and Space Complexity

- **Time Complexity:** O(m + n)
  Each element of both arrays is visited once.
- **Space Complexity:** O(m + n)
  A new array holds the merged result.

Program
18

Find the Frequency of Each Element

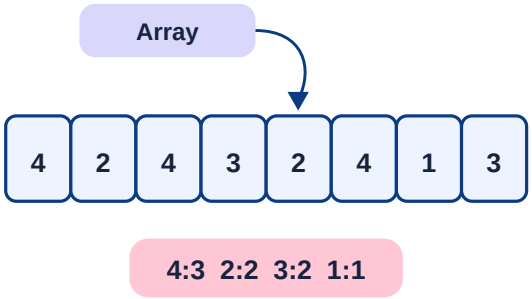
Topic
Hashing

Problem Statement

Given an array, count how many times each distinct element appears.

Approach

1. Create an empty dictionary freq.
2. For each element, increase its count: freq[x] = freq.get(x, 0) + 1.
3. After the loop, every key holds its total count.
4. Print each key with its count.



Python Program

```python
n = int(input())
arr = list(map(int, input().split()))

freq = {}
for x in arr:
    freq[x] = freq.get(x, 0) + 1

for key, count in freq.items():
    print(key, "->", count)
```

Input

```
8
4 2 4 3 2 4 1 3
```

Line 1: n | Line 2: array elements

Output

```
4 -> 3
2 -> 2
3 -> 2
1 -> 1
```

Dry Run / Illustration

Element	Its count now
4	1
2	1
4	2
3	1
2	2
4	3
1	1
3	2

Key Takeaways

- dict.get(key, 0) avoids a KeyError for new elements.
- collections.Counter(arr) does the same in one line.
- Frequency counting is the base of many hashing problems.

Time and Space Complexity

- **Time Complexity:** O(n)
  One pass to count.
- **Space Complexity:** O(k)
  k = number of distinct elements.

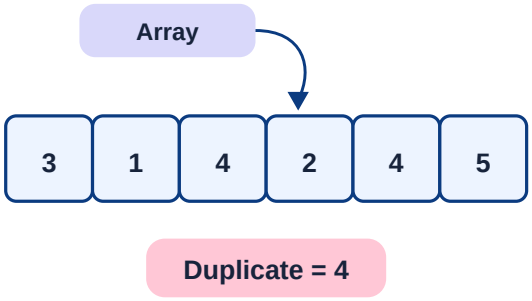
Program
19

Find a Duplicate Element

Topic
Hashing

Problem Statement

Given an array, find the first element that appears again (a duplicate).



Approach

1. Create an empty set called seen.
2. Traverse the array element by element.
3. If the element is already in seen, it is the duplicate - stop.
4. Otherwise add it to seen and continue.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

seen = set()
duplicate = -1
for x in arr:
    if x in seen:
        duplicate = x
        break
    seen.add(x)

print("Duplicate element:", duplicate)
```

Input

```
6
3 1 4 2 4 5
```

Line 1: n | Line 2: array elements

Output

```
Duplicate element: 4
```

Dry Run / Illustration

Element	In seen?	Action
3	No	add
1	No	add
4	No	add
2	No	add
4	Yes	duplicate!

Key Takeaways

- Set lookups are O(1) on average.
- Prints -1 when there is no duplicate.
- Trade-off: uses O(n) memory for speed.

Time and Space Complexity

- **Time Complexity:** O(n)
  Single traversal with O(1) lookups.
- **Space Complexity:** O(n)
  The set can store up to n items.

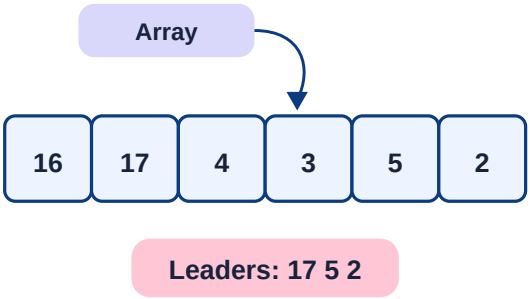
Program
20

Find the Leaders in an Array

Topic
Arrays

Problem Statement

An element is a leader if it is greater than every element to its right. Find all leaders.



Approach

1. The last element is always a leader.
2. Traverse from right to left keeping max_right.
3. If arr[i] is greater than max_right, it is a leader - update max_right.
4. Reverse the list to print leaders in original order.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

leaders = []
max_right = arr[n - 1]
leaders.append(max_right)
for i in range(n - 2, -1, -1):
    if arr[i] > max_right:
        max_right = arr[i]
        leaders.append(max_right)

leaders.reverse()
print("Leaders:", *leaders)
```

Input

```
6
16 17 4 3 5 2
```

Line 1: n | Line 2: array elements

Output

```
Leaders: 17 5 2
```

Dry Run / Illustration

i	arr[i]	Max right	Leader?
5	2	-	Yes
4	5	2	Yes
3	3	5	No
2	4	5	No
1	17	5	Yes
0	16	17	No

Key Takeaways

- Scanning from the right avoids a nested loop.
- Brute force is O(n^2); this is O(n).
- The rightmost element is always a leader.

Time and Space Complexity

- **Time Complexity:** O(n)
  One right-to-left pass.
- **Space Complexity:** O(n)
  The leaders list may hold up to n items.

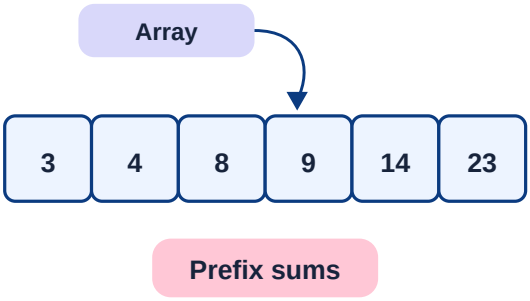
Program
21

Prefix Sum Array

Topic
Prefix Sum

Problem Statement

Given an array, build a prefix sum array where prefix[i] is the sum of arr[0..i].



Approach

1. Create prefix of the same size.
2. Set prefix[0] = arr[0].
3. For each i from 1, set prefix[i] = prefix[i - 1] + arr[i].
4. Print the prefix array.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

prefix = [0] * n
prefix[0] = arr[0]
for i in range(1, n):
    prefix[i] = prefix[i - 1] + arr[i]

print("Prefix sum:", *prefix)
```

Input

```
6
3 1 4 1 5 9
```

Line 1: n | Line 2: array elements

Output

```
Prefix sum: 3 4 8 9 14 23
```

Dry Run / Illustration

i	arr[i]	prefix[i-1]	prefix[i]
0	3	-	3
1	1	3	4
2	4	4	8
3	1	8	9
4	5	9	14
5	9	14	23

Key Takeaways

- Range sum from l to r = prefix[r] - prefix[l - 1].
- Turns repeated range-sum queries into O(1) each.
- itertools.accumulate(arr) builds it in Python.

Time and Space Complexity

- **Time Complexity:** O(n)
  One pass to build the array.
- **Space Complexity:** O(n)
  An extra array of size n.

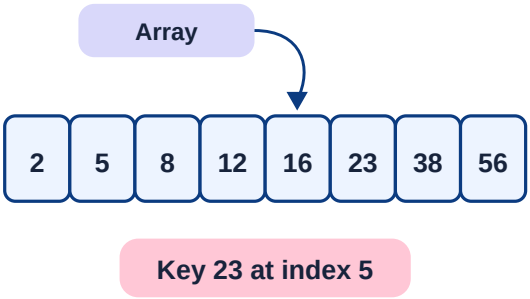
Program
22

Binary Search in a Sorted Array

Topic
Searching

Problem Statement

Given a sorted array and a key, find the index of the key using binary search.



Approach

1. Set low = 0 and high = n - 1.
2. Find mid = (low + high) // 2 and compare arr[mid] with the key.
3. If smaller, search the right half; if larger, search the left half.
4. Stop when found or when low > high.

Python Program

```python
n = int(input())
arr = list(map(int, input().split()))
key = int(input())

low, high = 0, n - 1
index = -1
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        index = mid
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1

if index != -1:
    print("Element found at index:", index)
else:
    print("Element not found")
```

Input

```
8
2 5 8 12 16 23 38 56
23
```

Line 1: n | Line 2: sorted array | Line 3: key

Output

```
Element found at index: 5
```

Dry Run / Illustration

Low	High	Mid	arr[mid]	Action
0	7	3	12	go right
4	7	5	23	found

Key Takeaways

- Halves the search space each step.
- The array MUST be sorted.
- Python offers bisect.bisect_left for this.

Time and Space Complexity

- **Time Complexity:** O(log n)
  The search range halves every iteration.
- **Space Complexity:** O(1)
  Only three index variables.

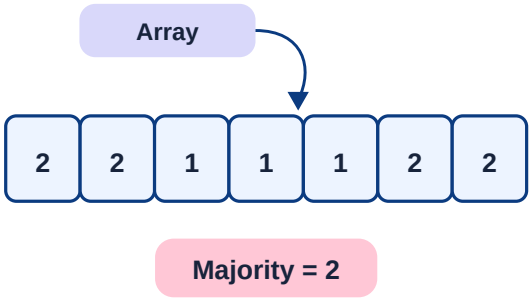
Program
23

Find the Majority Element (Boyer-Moore)

Topic
Voting

Problem Statement

Find the element that appears more than n / 2 times in the array, if one exists.



Approach

1. Keep a candidate and a count (start at 0).
2. If count is 0, take the current element as the candidate.
3. Add 1 to count if the element equals the candidate, otherwise subtract 1.
4. Verify that the candidate really appears more than n / 2 times.

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

candidate, count = None, 0
for x in arr:
    if count == 0:
        candidate = x
    count += 1 if x == candidate else -1

if arr.count(candidate) > n // 2:
    print("Majority element:", candidate)
else:
    print("No majority element")
```

Input

```
7
2 2 1 1 1 2 2
```

Line 1: n | Line 2: array elements

Output

```
Majority element: 2
```

Dry Run / Illustration

Element	candidate	count
2	2	1
2	2	2
1	2	1
1	2	0
1	1	1
2	1	0
2	2	1

Key Takeaways

- Pairs of different elements cancel each other out.
- The verification pass is needed when a majority may not exist.
- A dictionary solution works too but uses O(n) space.

Time and Space Complexity

- **Time Complexity:** O(n)
  One pass to vote, one to verify.
- **Space Complexity:** O(1)
  Only candidate and count are stored.

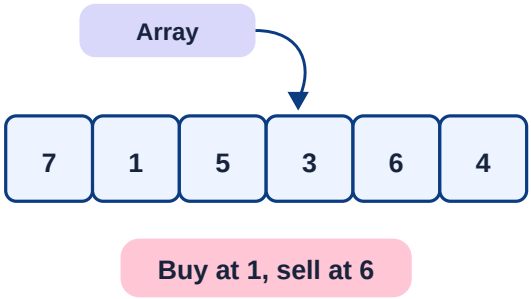
Program
24

Best Time to Buy and Sell Stock

Topic
Greedy

Problem Statement

Given daily stock prices, find the maximum profit from one buy followed by one later sell.



Approach

1. Track min_price, the lowest price seen so far.
2. For each new price, update min_price.
3. Profit if sold today = price - min_price.
4. Keep the maximum profit and print it.

Python Program

Python

```python
n = int(input())
prices = list(map(int, input().split()))

min_price = prices[0]
max_profit = 0
for price in prices[1:]:
    min_price = min(min_price, price)
    max_profit = max(max_profit, price - min_price)

print("Maximum profit:", max_profit)
```

Input

```
6
7 1 5 3 6 4
```

Line 1: n | Line 2: daily prices

Output

```
Maximum profit: 5
```

Dry Run / Illustration

Price	Min price	Profit	Max profit
1	1	0	0
5	1	4	4
3	1	2	4
6	1	5	5
4	1	3	5

Key Takeaways

- You must buy before you sell - track the minimum so far.
- A greedy one-pass solution beats the O(n^2) brute force.
- If prices only fall, the answer is 0.

Time and Space Complexity

- **Time Complexity:** O(n)
  Single pass through the prices.
- **Space Complexity:** O(1)
  Two variables only.

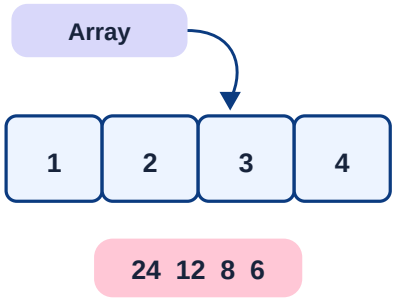
Program
25

Product of Array Except Self

Topic
Prefix Product

Problem Statement

For each index, find the product of all other elements - without using division.



Approach

1. Create result filled with 1.
2. Left pass: result[i] = product of all elements to the left of i.
3. Right pass: multiply result[i] by the product of all elements to the right.
4. result[i] now equals the product of all except arr[i].

Python Program

Python

```python
n = int(input())
arr = list(map(int, input().split()))

result = [1] * n
left = 1
for i in range(n):
    result[i] = left
    left *= arr[i]

right = 1
for i in range(n - 1, -1, -1):
    result[i] *= right
    right *= arr[i]

print("Result:", *result)
```

Input

```
4
1 2 3 4
```

Line 1: n | Line 2: array elements

Output

```
Result: 24 12 8 6
```

Dry Run / Illustration

Step	result
Left pass	1 1 2 6
Right i=3	1 1 2 6
Right i=2	1 1 8 6
Right i=1	1 12 8 6
Right i=0	24 12 8 6

Key Takeaways

- Combines a prefix product and a suffix product.
- Avoids division, so zeros in the array are fine.
- The output array does not count as extra space.

Time and Space Complexity

- **Time Complexity:** O(n)
  Two linear passes.
- **Space Complexity:** O(1)
  Extra space is constant (output excluded).